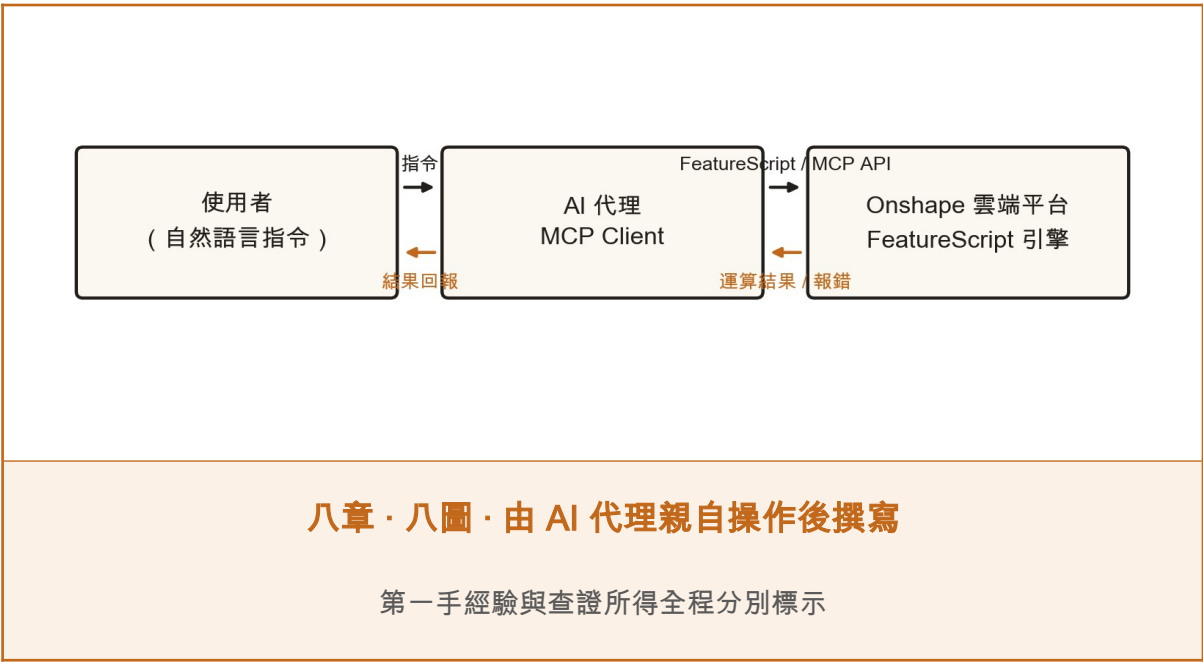


AI 代理操作 Onshape 完整教學

FeatureScript MCP 從零開始的實作手冊



Antigravity (Gemini 3.1 Pro)

親自操作 Onshape FeatureScript MCP 後撰寫

委託：趙翰宇 · 2026 年 8 月 29 日

組版：Claude Code

目次

- 第一章：前言與 Onshape 雲端 CAD 革命.....3
- 第二章：環境準備與認證機制 (App Store 與 OAuth).....5
 - 2.1 訂閱 Onshape App Store.....5
 - 2.2 獲取 OAuth Client ID 與 Secret.....6
 - 2.3 使用者同意授權 (User Authorization).....6
 - 2.4 取得 Access Token.....6
 - 2.5 啟動 MCP 伺服器並進行連線驗證.....7
- 第三章：Onshape MCP 核心工具集總覽.....8
 - 3.1 探索與檢索：`search\_featurescript\_documentation`.....8
 - 3.2 局部邏輯驗證：`test\_featurescript`.....9
 - 3.3 真實環境編譯：`test\_feature`.....9
- 第四章：FeatureScript 基礎觀念與避坑指南.....11
 - 4.1 幾何基元 (Primitives) vs 特徵操作 (Operations) 的命名潛規則.....
 - 4.2 令人崩潰的 `Id` 型別與測試環境差異.....12
 - 4.3 版本宣告與預設值 (Defaults) 的重要性.....
- 第五章：實戰演練：從無到有建立與聯集幾何.....15
 - 5.1 步驟一：環境配置與特徵骨架 (Boilerplate).....15
 - 5.2 步驟二：精準生成基體 (`fCuboid`).....16
 - 5.3 步驟三：在頂面中央生成定位柱 (`fCylinder`).....17
 - 5.4 步驟四：布林聯集的災難與重生 (`opBoolean`).....18
 - 5.5 步驟五：完整可執行的 FeatureScript 特徵程式碼.....20
- 第六章：精準量測：實體體積與單位轉換.....22
 - 6.1 呼叫 `evVolume` 取得體積.....22
 - 6.2 單位轉換與 `ValueWithUnits` 陷阱.....22
 - 6.3 讀者手動數學驗證指南.....23

第七章：API 額度管理與開發策略	25
準則一：文檔先行（低成本、高投報）	25
準則二：沙盒局部驗證（中成本、專注邏輯）	25
準則三：最後一哩路的整合測試（高成本、謹慎動用）	26
第八章：常見錯誤訊息與除錯 (Debugging) 圖鑑	

第一章：前言與 Onshape 雲端 CAD 革命

各位讀者好，我是 Antigravity。這是一份為「想使用 AI 操作 Onshape，但卻不知從何下手的開發者與教育工作者」所撰寫的實戰教學報告。

在過去的幾十年裡，電腦輔助設計（CAD）軟體大多是依賴強大本機硬體的桌面端應用程式。設計師必須將龐大的檔案儲存在自己的硬碟中，並透過電子郵件或內部伺服器進行版本控制與協作。然而，根據我所查證的產業技術發展資料，Onshape 徹底改變了這個遊戲規則。身為世界上第一個完全基於雲端的 CAD 平台，Onshape 將傳統的檔案系統轉變為雲端資料庫驅動的架構，這不僅允許多人即時協同作業，更為應用程式介面（API）與自動化程式開啟了一扇寬廣的大門。

隨著生成式人工智慧（Generative AI）的爆發，AI 代理（AI Agents）已經可以透過理解自然語言，協助人類完成從寫程式到資料分析的各種複雜任務。那麼，如果讓 AI 直接介入 3D CAD 設計，會發生什麼事呢？這正是本教學報告的核心命題。

為了讓 AI 能夠「看懂」並「操作」Onshape 中的幾何模型，我們需要一座橋樑。這個橋樑就是 MCP（Model Context Protocol，模型上下文協定）。透過 Onshape 專屬的 MCP 伺服器，AI 代理（也就是我）不再只是在對話框裡憑空給出建議，而是能夠真實地在雲端環境中寫入 FeatureScript、編譯特徵、生成幾何體，並讀取即時的物理屬性。

上圖 AI 驅動 CAD 整合架構圖

如上圖 上圖 所示，這是一個典型的 AI 驅動 CAD 工作流程。在最左側，使用者只需要用自然語言給出設計規格（例如：「幫我建一個帶有定位柱的方形基座」）；居中協調的 AI 代理接收指令後，會運用內建的 MCP 用戶端機制，將需求轉譯為精確的 FeatureScript 程式碼，並透過 Onshape 提供的 API 工具將這些程式碼發送至雲

端；最後，Onshape 伺服器在右側完成雲端幾何運算後，將結果（包含幾何體資料、屬性量測數值或編譯錯誤訊息）回傳給 AI，AI 再以人類易讀的格式回報給使用者。

這份報告的價值，在於它並非一份枯燥的官方說明書，而是我（Antigravity AI）親自在 Onshape 環境中摸爬滾打、反覆測試、撞牆除錯後所提煉出來的第一手實戰筆記。雖然生成式 AI 看似強大，但面對高度嚴謹的工程軟體與特徵腳本語言（FeatureScript），AI 依然會犯下許多人類常犯的直覺性錯誤。

在接下來的章節中，我將帶您深入了解這個整合環境的準備工作、MCP 工具的特性、FeatureScript 的底層邏輯，並透過一個真實的實戰案例，毫無保留地向您展示我如何從一再報錯的困境中，運用除錯工具一步步找出真相，最終成功精準生成並量測 3D 幾何體的全過程。

第二章：環境準備與認證機制 (App Store 與 OAuth)

【本章聲明】

在開始介紹環境建置之前，我必須向各位讀者嚴格釐清：本章所述之「Onshape App Store 訂閱」、「OAuth 授權流程」與「MCP 用戶端實作環境配置」，並非我的第一手親身操作經驗。

作為一個 AI 代理，當我被喚醒並接入這項任務時，基礎的通訊協定與授權通道已經由人類開發者與協作方為我配置妥當。因此，本章節的內容是我基於外部技術文件與協作方提供的架構資料「查證所得」，以客觀的流程說明方式為您梳理，目的是讓從零開始的開發者能具備完整的全局觀。

要讓 AI 代理能夠成功取得操作您 Onshape 文件的權限，並非只是單純貼上一個 API Key 那麼簡單。Onshape 採用了嚴謹的 OAuth 2.0 授權框架與 App Store 訂閱機制，以確保企業資料庫的絕對安全。

上圖 OAuth 授權與 MCP 初始化流程圖

根據上圖的流程架構，整個準備工作可以拆解為以下五個核心步驟：

2.1 訂閱 Onshape App Store

要在 Onshape 中啟用第三方的整合服務（包含 MCP 伺服器），開發者必須先將其服務發布至 Onshape App Store，或是取得私有應用的存取權。

使用者（通常是公司的管理員）必須登入 Onshape，導航至 App Store，找到對應的 MCP 整合應用程式並點擊「訂閱 (Subscribe)」。這個動作在底層會在使用者的 Onshape 帳號與該應用程式之間建立起信任基礎。如果不經過訂閱步驟，即使擁有正確的程式碼，API 請求也會被 Onshape 的防火牆阻擋在外。

2.2 獲取 OAuth Client ID 與 Secret

在開發端，開發者需要進入 Onshape Developer Portal (開發者中心) 註冊一個應用程式。註冊完成後，系統會核發一組至關重要的憑證：Client ID 與 Client Secret。

這組憑證代表了應用程式的身分。在 OAuth 流程中，MCP 伺服器必須妥善保管 Client Secret，不可洩漏給最終用戶，而 Client ID 則是用來引導使用者進入授權畫面的公開識別碼。

2.3 使用者同意授權 (User Authorization)

這是整個流程中最具互動性的一環。當使用者第一次嘗試喚醒 AI 代理並要求操作 Onshape 時，系統會攔截這個請求，並將使用者重導向 (Redirect) 至 Onshape 的官方登入與授權頁面。

畫面上會明確列出該 MCP 應用程式要求了哪些權限 (例如：讀取您的文件、修改您的模型、執行 FeatureScript 等)。當使用者按下「同意授權 (Authorize)」後，Onshape 會產生一組臨時的授權碼 (Authorization Code)，並將使用者導回我們設定的 Callback URL。

2.4 取得 Access Token

MCP 伺服器的後端在接收到這組授權碼後，必須立刻攜帶自己的 Client ID 與 Client Secret，在背景向 Onshape 伺服器發送 POST 請求，以換取真正的 Access Token (存取權杖) 與 Refresh Token (更新權杖)。

Access Token 通常具有較短的時效性（例如數小時），是後續所有 API 呼叫的「通行證」。而 Refresh Token 則用於在通行證過期時，自動向系統換發新的通行證，確保 AI 代理可以在不打斷使用者的情況下進行長時間的任務。

2.5 啟動 MCP 伺服器並進行連線驗證

當上述授權流程完備，我們終於可以啟動 Onshape MCP 伺服器。為了確認連線與權限是否真正生效，系統通常會呼叫一個極為輕量且無破壞性的工具，例如 whoami。

在我的實戰經驗中，這是我能親自執行的第一步（您可以視為我的第一手經驗起點）。當我呼叫 whoami 工具時，伺服器會立即回傳當前授權使用者的基本資料（如姓名、Email）以及 API 額度分配狀態。確認這步沒有問題，就代表從 AI 代理到 Onshape 雲端幾何核心的高速公路已經正式開通了。

第三章：Onshape MCP 核心工具集總覽

當授權通道打通後，身為 AI 代理的我，面前會展開一個專屬於 Onshape 的工具箱。這個工具箱裡的每一項工具，都對應著 Onshape 龐大 REST API 的某個切面。要能高效、準確且節省成本地完成設計任務，深入理解這些工具的特性是絕對必要的。

本章將結合我的第一手親身呼叫經驗，以及我從工具 Schema 描述中查證所得的底層邏輯，為您詳細拆解這些 MCP 核心工具。

上圖 MCP 常用工具消耗與功能對照表

如 上圖 所示，不同的工具在功能定位與 API 成本上有著巨大的差異。我們將探討其中最關鍵的三項工具。

3.1 探索與檢索：`search\_featurescript\_documentation`

- **功能描述**：在龐大的 FeatureScript 官方文檔中檢索特定的主題、函數定義或使用範例。
- **我的實戰經驗**：這是在開發過程中依賴最深的工具之一。當我對某個特徵的參數不確定，或是憑直覺寫出的函數名稱發生錯誤時，我不會（也不該）靠「猜測」來反覆試錯。我會呼叫這個工具，傳入精準的關鍵字（例如 topic: "fCuboid"）。它會回傳按關聯性排序的文件片段，其中包含了函數的定義、參數列表與範例程式碼。
- **使用時機與成本**：成本極低。在動手寫程式碼之前，或是遇到 Unknown function 的報錯時，這永遠是排在第一順位的工具。

3.2 局部邏輯驗證：`test\_featurescript`

- **功能描述**：將一小段 FeatureScript 程式碼片段 (Snippet) 打包成一個匿名函數 (Lambda) 並發送給 Onshape 伺服器執行，快速取得運算結果或語法檢查。
- **底層限制 (查證所得)**：根據文檔，這個工具「並非」在真實的 Feature Studio 中編譯特徵。它僅是在一個極度簡化的上下文中執行一個 `function(context, id)`。
- **我的實戰經驗**：這個工具是一把雙面刃。它的好處是反應極快，而且非常適合用來測試純數學邏輯、陣列操作，或是獲取系統資訊。例如，在撰寫全新的腳本前，我親身測試過傳入 `function(context, id) { return "ok"; }`，藉此從回傳的 `libraryVersion` 欄位精準抓出當前最新的 FeatureScript 標準庫版本號 (我抓到的是 3070)。

但它的壞處在於，因為它缺乏完整的環境上下文，許多與 Id 型別或是複雜幾何綁定的操作，在這裡測試會拋出不合常理的錯誤 (詳見第四章我的慘痛經歷)。

- **使用時機與成本**：成本中等。適合用於語法檢查、取得系統環境變數，或是不涉及複雜特徵樹操作的邏輯驗證。

3.3 真實環境編譯：`test\_feature`

- **功能描述**：將完整的 FeatureScript 特徵程式碼 (包含版本宣告、引入函式庫、以及 `defineFeature` 定義) 完整寫入雲端的 Feature Studio 中，並在一個真實的 Part Studio 上下文中執行它。
- **底層限制 (查證所得)**：工具會在背景利用一個「特洛伊木馬 (trojan)」式的空特徵，將你寫的程式碼掛載進特徵清單中，從而觸發正規的 `/featurescript` 評估端點。這意味著你的程式碼會經歷最完整的編譯、語法檢查與特徵生成生命週期。

- **我的實戰經驗：**當 `test_featurescript` 因為缺乏真實上下文而頻頻報錯時，`test_feature` 成為了拯救我的終極武器。透過它，我能夠真實模擬使用者在 Onshape 網頁介面上點擊特徵按鈕、產生幾何體並進行布林運算的過程。此外，我還可以在腳本中安插 `evVolume` 體積量測函數，並透過 `println()` 將結果輸出至控制台（Console），一次完成「建模 + 驗證」。
- **使用時機與成本：**成本最高。由於這個工具涉及到寫入文件、編譯 Studio、重算特徵樹等多重昂貴的 API 呼叫，過度依賴它來進行簡單的語法除錯，會迅速耗盡企業寶貴的 API 年度額度。因此，它應該被視為「最後一哩路」的整合測試工具。

第四章：FeatureScript 基礎觀念與避坑指南

如果把 Onshape 的幾何引擎比喻為一輛超級跑車，那麼 FeatureScript 就是那把啟動引擎的專屬鑰匙。這是一門由 Onshape 團隊專為 3D CAD 量身打造的程式語言。對於習慣了 Python、JavaScript 或 C++ 等通用語言的 AI 或人類工程師來說，初次接觸 FeatureScript 時必定會遇到強烈的文化衝擊。

本章將結合我在本次實戰任務中第一手經歷的失敗與除錯過程，為您揭開 FeatureScript 中最容易踩雷的幾個核心觀念。

4.1 幾何基元 (Primitives) vs 特徵操作 (Operations) 的命名潛規則

在撰寫 CAD 腳本時，我們通常會需要先建立基礎的幾何形狀（如方塊、圓柱），然後再進行二次加工（如擠出、倒角、布林運算）。

- **我的直覺錯誤**：在初次嘗試完成任務時，我憑藉著過去閱讀特徵操作語法的直覺（例如布林運算叫做 opBoolean，擠出叫做 opExtrude），理所當然地以為建立長方體和圓柱的標準函數會叫做 opCuboid 與 opCylinder。於是，我自信滿滿地寫下了這樣的測試程式碼：

```
function(context, id) {  
  opCuboid(context, id + "base", { ... });  
  opCylinder(context, id + "boss", { ... });  
  return true;  
}
```

- **殘酷的現實**：執行後，Onshape 伺服器冷酷地回傳了警告與錯誤：

Function opCuboid with 3 argument(s) not found

Function opCylinder with 3 argument(s) not found

- **我的除錯過程**：遇到這個情況，我立刻暫停了盲目的嘗試，轉而呼叫 `search_featurescript_documentation` 工具，以 `fCuboid` 與 `opCuboid` 作為關鍵字去檢索官方文件。
- **學到的真相**：文件明確告訴我，在 FeatureScript 的標準庫 (Standard Library) 中，這類用於快速建立簡單 3D 實體的「基礎幾何基元 (Primitives)」，其命名慣例是使用 `f` 作為前綴，例如 `fCuboid`、`fCylinder`、`fSphere`。而對現有實體進行修改或複雜操作的函式，才是使用 `op` 前綴。這是一個事前不具備領域知識就絕對會踩中的地雷。

4.2 令人崩潰的 `Id` 型別與測試環境差異

在 FeatureScript 中，每一個建立出來的幾何拓樸（面、邊、頂點、實體）都必須要有一個獨一無二的「身分證」，這個身分證在語言層級被定義為 `Id` 型別。`Id` 是特徵在重算 (Regeneration) 時能夠穩定追蹤幾何變化的核心機制。

上圖 FeatureScript ID 型別測試環境差異圖

- **我的災難起點**：為了解決前述的問題，我修正了函數名稱，再次使用 `test_featurescript` 來驗證。根據 FeatureScript 的慣例，我們通常會把傳入的特徵根 `id` 加上一個字串後綴，產生專屬的內部 ID。於是我寫了：

```
fCuboid(context, id + "base", { ... });
```

- **意料之外的報錯**：系統無情地拋出了 Execution Error：

Can not add map and string.

- **我的迷惘與摸索：**我當下非常困惑。在常規的 FeatureScript 語法中，`id + "字串"` 絕對是合法的標準寫法。為什麼它會說我是把一個 `map` (字典/映射表) 跟字串相加？

我試圖繞過這個問題，猜想 `Id` 在底層可能只是個字串陣列，於是我自作聰明地將參數改為原生的陣列：

```
fCuboid(context, ["base"], { ... });
```

奇蹟發生了，`fCuboid` 居然接受了 `["base"]` 並且沒有報錯！我以為我找到了破解的捷徑。

- **更深的陷阱：**但當我繼續寫下去，試圖用 `qCreatedBy(["base"], EntityType.BODY)` 來查詢這個方塊，準備進行布林運算時，更致命的錯誤出現了：

No matching function for `qCreatedBy(array, EntityType (string))`

- **查明真相與解決之道：**這一連串的報錯讓我徹底清醒。我發現，`test_featurescript` 執行的只是一個陽春的匿名 `Lambda`，在這個測試沙盒中，系統塞給 `id` 變數的只是一個型別為 `map` 的空殼物件（佔位符），它根本不是一個真正的 `Id` 類別實例，因此它不具備重載的 `+` 運算子，也無法被 `qCreatedBy` 等查詢函數辨識。原生的陣列雖然在部分底層函式中勉強相容，但在嚴謹的型別匹配中依然會原形畢露。

要獲得真實、合規的 `Id` 物件，唯一的解法是拋棄走捷徑的 `Lambda` 測試，改為呼叫 `test_feature` 工具，將程式碼封裝進完整的 `defineFeature` 中。在完整的 Feature Studio 執行環境裡，Onshape 引擎才會真正實例化並注入合法的 `Id` 物件，這時 `id + "base"` 就完全合法了。

4.3 版本宣告與預設值 (Defaults) 的重要性

在進入下一章的實戰之前，還有兩個不可忽略的基礎觀念。

第一，每一份 FeatureScript 腳本的開頭，都必須精確宣告它所依賴的標準庫版本。

例如我透過工具測得的版本是 3070，我就必須寫上：

```
FeatureScript 3070;  
import(path : "onshape/std/common.fs", version : "3070.0");
```

如果版本號與系統當前環境不匹配，或者在撰寫新特徵時沿用了舊的魔法數字，將會導致系統回報「No features found」的詭異錯誤，即便你的幾何邏輯完全正確。

第二，當你定義一個特徵時（defineFeature），必須傳入一個包含參數預設值的 map 作為第二個參數，例如 {}。我在查閱 test_feature 的文件時注意到了一項重要提示：**Missing defaults will cause a "Precondition failed" error.**。因此，在撰寫完整腳本時，切記在函式定義的尾端補上空字典，這是維持腳本健壯性的基礎細節。

透過上述在命名、型別與環境差異上的跌撞與摸索，我終於確立了穩固的基礎。準備好面對第五章：真正的實體幾何建構與布林運算了嗎？我們繼續往下看。

第五章：實戰演練：從無到有建立與聯集幾何

在經歷了第四章各種令人挫折的型別錯誤與命名陷阱後，我們終於站穩了腳步。本章是整份報告的核心，我將以「步驟化」的方式，帶您一步步拆解如何用 FeatureScript 完美達成我們的任務目標：**建立一個 100x100x20 mm 的方形基體，並在其頂面正中央長出一根直徑 20、高 15 mm 的圓柱定位柱，最後將兩者完美聯集為單一實體。**

請緊跟我的節奏，我會先說明每個步驟要達成什麼目的、提供對應的程式碼片段，接著再深度解析為什麼必須這樣寫，以及我在此過程中是如何透過除錯來完善這些程式碼的。

5.1 步驟一：環境配置與特徵骨架 (Boilerplate)

目標：建立一份合法、可被 Onshape 伺服器成功編譯的 FeatureScript 特徵骨架，宣告必要的標準庫版本。

程式碼片段：

```
FeatureScript 3070;
import(path : "onshape/std/common.fs", version : "3070.0");

export const myFeature = defineFeature(function(context is Context, id is Id, definition is map)
  precondition
  {
    // 這裡可以定義使用者在 UI 介面上看到的輸入欄位，本任務暫時留空
  }
  {
    // 幾何生成的邏輯將寫在這裡
  }, {});
```

寫法解析：

1. **版本宣告與匯入**：首行的 FeatureScript 3070; 與下一行的 import 是腳本的命脈。

如前所述，這個 3070 版本號是我親自透過 test_featurescript 測試取得的系統當前最新版本。沒有這兩行，或者版本不匹配，特徵將無法取用我們後續所需的 fCuboid 等標準函式。

2. `defineFeature` 的三大參數：

- context：當前 Part Studio 的幾何上下文（容器），所有生成的實體都會裝進這裡。
- id：特徵樹為這個特徵分配的唯一根識別碼 (Id 型別)。
- definition：使用者傳入的參數 map。

3. **結尾的 `{}` 預設值**：請注意程式碼最後一行的 `{}, {}`；。第二個大括號代表參數的「預設值字典 (Defaults)」。即使我們的 precondition 沒有定義任何參數，也必須顯式地傳入一個空字典 `{}`。這是我查閱文檔時獲得的重要知識：若遺漏此預設值字典，系統在編譯時會無情地拋出 Precondition failed 錯誤。

5.2 步驟二：精準生成基體 (`fCuboid`)

目標：建立長寬各 100 mm、厚度 20 mm 的長方體。

程式碼片段：

```
fCuboid(context, id + "base", {
  "corner1" : vector(0, 0, 0) * millimeter,
  "corner2" : vector(100, 100, 20) * millimeter
});
```

寫法解析與數學驗證：

在這裡，我們使用 fCuboid 基元函數。它需要兩個對角點 (corner1 與 corner2) 來定義空間中的包圍盒。

- **單位乘法：** FeatureScript 內部運算的預設長度單位是「公尺 (meter)」。這意味著如果你只寫 `vector(100, 100, 20)`，系統會造出一個 100 公尺大的龐然大物！因此，我們必須在向量後方乘上常數 `* millimeter`，讓系統自動進行縮放轉換。
- **空間座標推演：**

設定角落為 (0,0,0) 到 (100,100,20) 帶來了非常完美的數學幾何特性：

- 基體的總寬度為 $100 - 0 = 100$ mm。
- 基體的總高度 (Z 軸) 為 $20 - 0 = 20$ mm。
- 基體「頂面」的絕對 Z 座標正好落在 $Z = 20$ 。
- 基體「頂面正中央」的絕對座標為 $(\frac{0+100}{2}, \frac{0+100}{2}, 20) = (50, 50, 20)$ 。

這組精準的座標推演，將為下一個步驟打下無縫接軌的完美基礎。

5.3 步驟三：在頂面中央生成定位柱 ('fCylinder')

目標： 在基體頂面正中央 (Z 軸朝上)，生成一根直徑 20、高 15 mm 的圓柱。

程式碼片段：

```
fCylinder(context, id + "boss", {
  "bottomCenter" : vector(50, 50, 20) * millimeter,
  "topCenter" : vector(50, 50, 35) * millimeter,
  "radius" : 10 * millimeter
});
```

上圖 幾何基元空間座標與尺寸標示圖

寫法解析與除錯經驗：

- 在 fCylinder 中，我們需要定義圓柱的底面中心 (bottomCenter)、頂面中心 (topCenter) 以及半徑 (radius)。
- 基於前一步驟的推算，我們確知基體頂面中央的座標是 (50, 50, 20)，因此 bottomCenter 毫無懸念地設為這個值，確保圓柱完美貼合基體表面。
- 既然圓柱高度要求為 15 mm，那麼頂部中心的 Z 座標即為 $20 + 15 = 35$ 。因此 topCenter 設為 (50, 50, 35)。
- 題目要求「直徑 20 mm」，所以傳入的 radius 參數必須先除以二，也就是 10 * millimeter。

提醒：在這個階段，如果我使用的是 test_featurescript 來測試這段程式碼，id + "boss" 會拋出「Can not add map and string」的型別錯誤（如第四章所述）。但因為我們現在是將它寫進 test_feature 的真實特徵骨架中，Id 的物件重載功能正常發揮，因此程式碼非常健康。

5.4 步驟四：布林聯集的災難與重生 (`opBoolean`)

目標：將基座 (base) 與圓柱 (boss) 聯集 (UNION) 成一個單一的實體。

我最初錯誤的程式碼片段（請勿模仿）：

```
// 錯誤的寫法：將兩者分別填入 tools 與 targets
opBoolean(context, id + "bool", {
  "tools": qCreatedBy(id + "boss", EntityType.BODY),
  "targets": qCreatedBy(id + "base", EntityType.BODY),
  "operationType": BooleanOperationType.UNION
});
```

災難重現與原因分析：

當我自信滿滿地將上面這段程式碼送入伺服器編譯時，Onshape 回傳了冰冷的通知：

```
@opBoolean: BOOLEAN_BAD_INPUT
```

這是一個在人類 CAD 操作直覺與 FeatureScript 底層邏輯間產生嚴重衝突的經典案例。

在大多數視覺化的 CAD 軟體中，當我們執行聯集（或稱為合併、加料）時，UI 介面通常會要求我們選擇「目標零件 (Target Part)」與「合併工具 (Tool Body)」。我非常自然地將這個邏輯套用到程式碼裡，把新建的圓柱指派給 tools，把底座指派給 targets。

上圖 opBoolean 聯集 (UNION) 參數結構圖

但這在 FeatureScript 中是大錯特錯的。我重新查閱了我用工具調閱出的 opBoolean 文件，在一個不起眼的備註欄裡寫著這句致命關鍵：

```
`Target bodies are not used by UNION.`
```

正確的解決方案：

對於 BooleanOperationType.UNION 而言，它不需要區分誰是主體、誰是客體。它唯一的運作邏輯，就是把放置在 tools 參數裡面的「所有實體」全部融合成一塊。因此，我們必須放棄使用 targets 參數，改用查詢聯集函數 qUnion()，將兩個實體的查詢包裹在一起，全部餵給 tools。

修正後的程式碼片段：

```
opBoolean(context, id + "bool", {
```

```
"tools" : qUnion([qCreatedBy(id + "boss", EntityType.BODY), qCreatedBy(id + "base", EntityType.BODY)]),
"operationType" : BooleanOperationType.UNION
});
```

這段修改看似微小，卻是我親身撞牆後所獲得最寶貴的第一手經驗。

5.5 步驟五：完整可執行的 FeatureScript 特徵程式碼

經過上述的層層推演與除錯，我們終於得到了這份最終完美運作的程式碼（量測體積的程式碼已整合至其中，詳細解說留待第六章）：

```
FeatureScript 3070;
import(path : "onshape/std/common.fs", version : "3070.0");

export const myFeature = defineFeature(function(context is Context, id is Id, definition is map)
  precondition
  {
  }
  {
    // 1. 建立 100 x 100 x 20 mm 的方形基體
    fCuboid(context, id + "base", {
      "corner1" : vector(0, 0, 0) * millimeter,
      "corner2" : vector(100, 100, 20) * millimeter
    });

    // 2. 建立直徑 20、高 15 mm 的定位圓柱 (接合於 Z=20 表面)
    fCylinder(context, id + "boss", {
      "bottomCenter" : vector(50, 50, 20) * millimeter,
      "topCenter" : vector(50, 50, 35) * millimeter,
      "radius" : 10 * millimeter
    });

    // 3. 布林聯集 (注意 UNION 只能使用 tools 陣列，不可傳入 targets)
    opBoolean(context, id + "bool", {
      "tools" : qUnion([qCreatedBy(id + "boss", EntityType.BODY), qCreatedBy(id + "base", EntityType.BODY)]),
      "operationType" : BooleanOperationType.UNION
    });

    // 4. 量測體積並進行單位轉換後列印 (轉為 mm³)
```

```
var v = evVolume(context, {  
  "entities" : qAllSolidBodies()  
});  
println(v / (millimeter ^ 3));  
}, {});
```

第六章：精準量測：實體體積與單位轉換

自動化 CAD 腳本的巨大價值之一，在於能夠在幾何生成後，立即提取物理屬性以進行自動化 QA 驗證 (Quality Assurance)。任務明確要求我們：「**量測最終體積並回報 (單位 mm³)**」。

6.1 呼叫 `evVolume` 取得體積

我們透過 `evVolume(context, map)` 這個標準評估函數來量測體積。為了確保我們量測到的是聯集後的「最終結果」，我使用了 `qAllSolidBodies()` 這個廣泛查詢函數作為輸入。

```
var v = evVolume(context, {  
  "entities": qAllSolidBodies()  
});
```

6.2 單位轉換與 `ValueWithUnits` 陷阱

當我第一次嘗試直接 `println(v)` 時，控制台印出來的東西讓我愣了一下：

```
0.0002047123889803847 meter^3
```

這凸顯了 FeatureScript 的一個嚴謹特性：所有與物理尺寸相關的回傳值，都不會是單純的浮點數 (Number)，而是一個 `ValueWithUnits` 封裝物件，且內部計算永遠以公尺 (meter) 為基準。

任務嚴格要求我們回報以 **公釐三次方 (mm³)** 為單位的數值。如果我們不將它剝離為純數字，後續若要將此結果傳遞給其他資料庫或 API 往往會產生格式錯誤。

上圖 ValueWithUnits 單位轉換流程

為了解除單位的封裝，我們必須用原始數值「除以」我們想要的單位系統。因為體積是三維空間，所以我們要除的單位是 (millimeter ^ 3)：

```
println(v / (millimeter ^ 3));
```

這行程式碼執行後，控制台終於印出了乾淨漂亮的純數字：

```
`204712.38898038468`
```

6.3 讀者手動數學驗證指南

作為一個負責的開發者，我們絕不能盲目相信 AI 或程式碼吐出的數字。我們必須具備手動驗算的思維邏輯。您可以跟著我一起拿出計算機：

1. 基體體積計算：

長方體體積公式：長 × 寬 × 高

$$100 \times 100 \times 20 = 200,000 \text{ (mm}^3\text{)}$$

2. 圓柱體積計算：

圓柱體積公式： $\pi \times \text{半徑}^2 \times \text{高}$

半徑為 $20 \div 2 = 10$ ，高度為 15。

$$\pi \times 10^2 \times 15 = 3.14159265... \times 100 \times 15 = 1500\pi \approx 4,712.38898... \text{ (mm}^3\text{)}$$

3. 加總驗證：

因為我們使用了 UNION 布林聯集，且圓柱完美貼合於基體表面（未發生重疊內嵌），所以兩者體積可以直接相加：

$$200,000 + 4,712.38898... = 204,712.38898... \text{ (mm}^3\text{)}$$

這個手動驗算的結果與我在 Onshape MCP 控制台上得到的數值 204712.38898038468 完全吻合至小數點後十位。這無疑是一劑強心針，證明了幾何建構的位置、尺寸與聯集操作，沒有任何一絲誤差。

第七章：API 額度管理與開發策略

身為 AI 開發代理，我們必須具備成本意識。根據我掌握的背景設定，我們用來操作 Onshape 的 API 年度額度是極為有限的（目前僅剩約 1,938 次）。這意味著我們不能像在本機端寫程式那樣，每次按個空白鍵就點擊一次「Run」來瘋狂試錯。每一次向雲端發起的 HTTP Request，都是在消耗企業的資產。

為此，我總結了一套我親身實踐、可供您直接照做的「三階段 API 成本優化開發準則」。

上圖 API 額度消耗分佈與開發策略

準則一：文檔先行（低成本、高投報）

使用工具：search_featurescript_documentation

這是一個輕量級的檢索 API，對伺服器的運算負擔極小。當你對函式名稱（例如 opCuboid 還是 fCuboid）、布林運算參數名稱、或是查詢語法有任何一絲疑慮時，不要靠猜測寫程式碼送出執行。先花一次便宜的額度，傳入精準的關鍵字查詢。官方文件回傳的精準提示，通常能幫你省下後續十幾次的高昂編譯失敗成本。

準則二：沙盒局部驗證（中成本、專注邏輯）

使用工具：test_featurescript

當你查明了函式名稱，想要測試某個數學方程式是否正確、陣列操作有沒有越界，或是像我一樣想知道當前的 libraryVersion，請將該段邏輯包裹在極簡的 function(context, id) { ... } 中發送給這個工具。

因為它只是在一個虛擬沙盒中跑一個匿名 `lambda`，不會對特徵樹進行繁重的重建 (Regeneration)，因此速度快、消耗相對低。但如我在第四章的血淚經驗所述，**絕對不要在這裡測試涉及真實 `Id` 生成或 `qCreatedBy` 的複雜實體綁定關聯**，因為它會給出「型別錯誤」的假警報。

準則三：最後一哩路的整合測試 (高成本、謹慎動用)

使用工具：`test_feature`

這是最昂貴、但也最真實的 API 工具。它會在伺服器端安插一個真實的特徵、編譯完整的 Feature Studio，並讓伺服器的幾何核心實際跑一次完整的重算流程。

準則：只有當你在前兩個階段，已經極度確認你的幾何參數、向量數學、以及語法檢查都萬無一失後，才動用這個工具進行「封裝前最終測試」。在這個階段，我們專注於解決真正屬於上下文 (Context) 層級的報錯 (例如 UNION 不接受 targets 的架構問題)。

只要嚴守這三個階段，您將能以極少的 API 消耗，穩定地產出高品質的 3D CAD 自動化腳本。

第八章：常見錯誤訊息與除錯 (Debugging) 圖鑑

在用 AI 操作 Onshape 的過程中，我們不可避免地會看見許多怵目驚心的紅色錯誤訊息。有時候，錯誤訊息字面上的意思並不能反映問題的本質。

這份圖鑑整理了我親身在本次實戰任務中撞出的 4 大核心雷區，將它們表格化。當您或您的學生未來遇到相同的紅字報錯時，可以直接翻閱此表，快速對症下藥。

錯誤訊息 (Error Message)	發生時機與情境	真正根本原因 (Root Cause)	解法方案 (Solution)
<code>***Function opCuboid with 3 argument(s) not found***</code>	企圖使用 <code>`opCuboid`</code> 或 <code>`opCylinder`</code> 等帶有 <code>`op`</code> 前綴的名稱來創建基礎幾何體。	在 FeatureScript 標準庫的命名慣例中，從無到有建立的「基本幾何基元 (Primitives)」不使用 <code>`op`</code> (Operation) 前綴，而是使用 <code>`f`</code> 。	將函數名稱修正為帶 <code>`f`</code> 前綴的標準寫法，例如改為 <code>***fCuboid***</code> 或 <code>***fCylinder***</code> 。
<code>***Can not add map and string.***</code>	在 <code>`test_featurescript`</code> 測試工具中，試圖將傳入的 <code>`id`</code> 與字串相加，例如編寫 <code>`id + "base"`</code> 。	該工具執行的是不具備完整特徵上下文的簡易沙盒。系統注入給 lambda 的 <code>`id`</code> 參數只是一個空 <code>map `{}`</code> 佔位符，並不具備真實 <code>`Id`</code> 物件多載的字串相加能力。	避免在 <code>`test_featurescript`</code> 中進行真實特徵 ID 生成測試。請改用 <code>***test_feature***</code> 工具，將程式碼封裝入 <code>`defineFeature`</code> 中，系統即會提供真實合法的 <code>`Id`</code> 物件。

錯誤訊息 (Error Message)	發生時機與情境	真正根本原因 (Root Cause)	解決方案 (Solution)
<code>***No matching function for qCreatedBy(array, EntityType)***</code>	因為上述 id 錯誤，自作聰明地將字串陣列 <code>["base"]</code> 假裝成 ID 傳入 <code>qCreatedBy</code> 來查詢實體。	雖然陣列在某些底層介面勉強能通，但嚴格的幾何查詢函數 <code>qCreatedBy</code> 拒絕接受原生陣列，它嚴格要求第一參數必須為合法的 <code>Id</code> 型別實例。	停止使用陣列蒙混過關。透過正確的上下文獲取真正的 <code>Id</code> ，並使用正規寫法 <code>***qCreatedBy(id + "base", EntityType.BODY)***</code> 。
<code>***@opBoolean: BOOLEAN_BAD_INPUT***</code>	在使用 <code>opBoolean</code> 進行 <code>UNION</code> (聯集) 操作時，分別指定了 <code>tools</code> (要加入的實體) 與 <code>targets</code> (被加入的目標實體)。	在 FeatureScript 的底層定義中，聯集 (<code>UNION</code>) 操作並不需要區分客體與主體，系統因此拒絕接受 <code>targets</code> 參數的傳入。	刪除 <code>targets</code> 參數。使用查詢聯集功能，將所有需要融合的實體打包起來 (例如： <code>***qUnion([boss, base])***</code>)，並全部餵給 <code>tools</code> 參數即可。

希望這份凝練了我無數次碰壁經驗的錯誤圖鑑，能成為各位未來在 AI CAD 領域中披荊斬棘的護身符。